Linux for Beginners

A Practical Guide

Chapter 4 of 12

Users, Permissions & the Root User

Contents

Chapter 4

Users, Permissions & the Root User

4.1 Linux Identity: Users and Groups

Linux was designed from the ground up as a **multi-user system**. Even if you're the only person using your computer, Linux still thinks in terms of multiple users — because the operating system itself runs many processes under different identities to keep things secure and isolated.

Every file, process, and resource in Linux belongs to a **user** and a **group**. Understanding this identity system is the key to understanding permissions.

Users

Every person (or service) that interacts with a Linux system has a **user account**. Each account has:

- A unique **username** (e.g. `ashik`)
- A unique numeric **UID** (User ID) — Linux uses this number internally; the username is a human-friendly label
- A **home directory** — typically `/home/username`
- A **default shell** — usually `/bin/bash`

Groups

Groups are collections of users. Every user belongs to at least one group — their **primary group** (usually the same name as the username). Users can also belong to additional **supplementary groups**, which grants access to shared resources.

For example, on Ubuntu, members of the `sudo` group can run administrator commands. Members of the `docker` group can manage containers. This group-based access control is powerful and flexible.

Viewing your identity

```bash
$ whoami
ashik

$ id
uid=1000(ashik) gid=1000(ashik) groups=1000(ashik),4(adm),27(sudo),1001(docker)

$ id ashik          # look up another user
uid=1000(ashik) gid=1000(ashik) groups=1000(ashik),27(sudo)
```

The `id` command shows your UID, primary GID, and all the groups you belong to. This is the fastest way to check what access you have.

4.2 The Root User and the Principle of Least Privilege

Linux has one special user above all others: **root**. The root user (UID 0) has unrestricted access to everything on the system — every file, every process, every configuration. Root can delete system files, change any password, install software globally, and override every permission check.

This power is why you should almost never log in directly as root. Instead, Linux follows the **principle of least privilege**: run with only the access you need for the current task, and no more.

Why root is dangerous

- A typo in a root terminal can destroy the entire system — there are no safety checks
- Malware or a compromised process running as root has total control over the system
- It's very easy to accidentally overwrite critical system files with no warning
- Actions as root leave no margin for error — there is no undo

> **TIP**
> On modern Ubuntu and most distributions, the root account is locked by default. You are not meant to log in as root — you're meant to use sudo for individual privileged tasks.

4.3 sudo: Running Commands as Root Safely

`sudo` (**s**uper**u**ser **do**) lets you run a single command with root privileges, then immediately drops back to your regular user account. It's the standard, safe way to do administrative work.

```bash
# Without sudo — permission denied:
$ apt update
E: Could not open lock file /var/lib/dpkg/lock-frontend (13: Permission denied)

# With sudo — works:
$ sudo apt update
[sudo] password for ashik:
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
...
```

How sudo works

1. You type `sudo` before the command

2. Linux checks if your account is in the `sudo` group (or has a sudoers rule)
3. If yes, it asks for **your own password** (not root's)
4. The command runs as root
5. Your session returns to normal immediately after

Essential sudo patterns

```bash
$ sudo command                  # run a single command as root
$ sudo -i                       # open a root shell (use sparingly)
$ sudo -u username command      # run as a specific user
$ sudo !!                       # re-run previous command with sudo
$ sudo -l                       # list your sudo permissions
$ sudo -k                       # clear cached credentials immediately
```

The sudoers file

Who can use `sudo` is controlled by `/etc/sudoers`. Never edit it directly — always use `visudo`, which validates syntax before saving (a syntax error in sudoers can lock you out):

```bash
$ sudo visudo                   # safely edit the sudoers file
```

> **TIP**
> sudo caches your credentials for 15 minutes by default. After that it will ask for your password again. Clear the cache manually with: sudo -k

---

4.4 Understanding the Permission Model

Every file and directory in Linux has a set of permissions that controls who can do what with it. The model is elegantly simple: three **permission types**, applied to three **categories of people**.

The three permission types

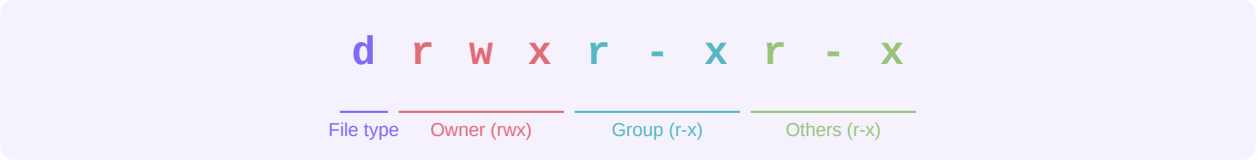
Symbol	Name	On a file	On a directory
r	Read	View the file's contents	List the directory's contents (ls)
w	Write	Modify or delete the file	Create, delete, or rename files inside
x	Execute	Run the file as a program	Enter the directory (cd into it)
-	None	Permission is not granted	Permission is not granted

The three categories

Category	Who it applies to
Owner (u)	The user who owns the file — usually whoever created it
Group (g)	Members of the file's associated group
Others (o)	Everyone else — any user not in the above two categories

4.5 Reading Permission Strings

When you run `ls -l`, the first column is the permission string. Here's how to decode it:



Breaking down real examples:

```bash
$ ls -l
-rw-r--r--  1 ashik ashik  1420 Jun 10 09:21 notes.txt
drwxr-xr-x  2 ashik ashik  4096 Jun 10 09:15 Documents
-rwxr-xr-x  1 ashik ashik   512 Jun 10 08:00 script.sh
-rw-------  1 ashik ashik  2048 Jun 10 07:55 private.key
```

String	Type	Owner	Group	Others	Meaning
-rw-r--r--	file	rw-	r--	r--	Owner read/write; group and others read-only
drwxr-xr-x	dir	rwx	r-x	r-x	Owner full control; others can list and enter
-rwxr-xr-x	file	rwx	r-x	r-x	Owner full; others can read and execute
-rw-------	file	rw-	---	---	Only owner can read/write — completely private

4.6 Changing Permissions with chmod

`chmod` (**ch**ange **mod**e) changes the permissions of a file or directory. There are two ways to use it: **symbolic mode** (human-readable) and **numeric (octal) mode** (compact and precise).

Symbolic mode

Symbolic mode uses letters: **who** (u/g/o/a), **operation** (+/-/=), and **what** (r/w/x).

```bash
$ chmod u+x script.sh          # add execute for owner
$ chmod g-w file.txt           # remove write from group
$ chmod o+r file.txt           # add read for others
$ chmod a+x script.sh          # add execute for all (a = all)
$ chmod u+x,g-w file.txt       # multiple changes at once
$ chmod u=rw,go=r file.txt     # set exactly (= replaces entirely)
```

Numeric (octal) mode

Each permission has a numeric value. Add them up for each category to get a 3-digit number. This is the most common way permissions appear in documentation and scripts.

Permission	Value
read (r)	4
write (w)	2
execute (x)	1
none (-)	0

Common octal permission sets:

```bash
$ chmod 755 script.sh     # rwxr-xr-x  standard executable
$ chmod 644 notes.txt     # rw-r--r--  standard file
$ chmod 600 private.key   # rw-------  private file
$ chmod 700 my_scripts/   # rwx------  private directory
$ chmod 777 shared/       # rwxrwxrwx  world-writable (avoid!)
$ chmod -R 755 project/   # apply recursively to all contents
```

Octal	String	Typical Use
777	rwxrwxrwx	Everyone full access — avoid except specific shared dirs
755	rwxr-xr-x	Standard for executables and public directories
644	rw-r--r--	Standard for regular files (documents, configs)
700	rwx------	Private directory — only owner can access
600	rw-------	Private file — SSH keys, secrets
664	rw-rw-r--	Owner and group can edit; others read-only

> **TIP**
> SSH will refuse to use a private key if its permissions are too open. Always set chmod 600 ~/.ssh/id_rsa — if permissions are wrong, SSH will refuse to connect.

4.7 Changing Ownership with chown and chgrp

Permissions only mean something in context of who *owns* the file. `chown` changes the owner (and optionally the group). `chgrp` changes just the group. Both typically require `sudo`.

```bash
# Change owner only
$ sudo chown root notes.txt         # change owner to root
$ sudo chown ashik notes.txt        # change back to ashik

# Change owner and group together
$ sudo chown ashik:developers app.py

# Change group only
$ sudo chgrp developers app.py

# Recursive — apply to entire directory tree
$ sudo chown -R ashik:ashik /home/ashik/
```

> **TIP**
> You can only chown files you own (changing to another owner requires sudo). You can chgrp a file you own to any group you belong to — no sudo required.

4.8 Special Permissions: setuid, setgid, and the Sticky Bit

Beyond the standard rwx model, Linux has three special permission bits for specific situations you'll encounter on real systems.

Bit	Symbol	On Files	On Directories
setuid (4000)	s in owner exec slot	File runs as its owner — e.g. passwd runs as root so any user can change their password	Rarely used on directories

Bit	Symbol	On Files	On Directories
setgid (2000)	s in group exec slot	File runs with the group's identity	New files inherit the directory's group — useful for shared project dirs
Sticky bit (1000)	t in others exec slot	Rarely used on files	Only file owner can delete their own files — used on /tmp

```bash
# setuid on passwd — note the 's' in owner execute position:
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 59976 Feb  6 2022 /usr/bin/passwd

# sticky bit on /tmp — note the 't' in others execute position:
$ ls -ld /tmp
drwxrwxrwt 18 root root 4096 Jun 10 09:00 /tmp

# Set setgid on a shared directory:
$ chmod g+s shared_project/
$ chmod 2775 shared_project/     # numeric: 2 = setgid
```

4.9 Managing Users and Groups

On a single-user desktop you may not manage users often, but these commands are essential for server administration and will appear frequently on real Linux systems.

User management

```bash
$ sudo adduser ashik2             # create a new user (interactive, recommended)
$ sudo useradd -m -s /bin/bash newuser  # lower-level, non-interactive
$ sudo passwd ashik2               # set or change a user's password
$ sudo usermod -aG sudo ashik2     # add user to the sudo group
$ sudo usermod -aG docker ashik    # add yourself to docker group
$ sudo deluser ashik2              # delete a user (keeps home dir)
$ sudo deluser --remove-home ashik2  # delete user and home directory
```

Group management

```bash
$ sudo groupadd developers        # create a new group
$ sudo groupdel developers        # delete a group
$ sudo gpasswd -a ashik developers  # add user to group
$ sudo gpasswd -d ashik developers  # remove user from group
$ groups ashik                     # list all groups for a user
$ cat /etc/group                   # view all groups on the system
```

Key system files

File	Contains
/etc/passwd	User account info: username, UID, home dir, shell (no passwords)
/etc/shadow	Hashed passwords — readable only by root
/etc/group	Group definitions: name, GID, member list
/etc/sudoers	Rules for who can use sudo and how

> **TIP**
> After adding yourself to a new group with usermod -aG, you must log out and back in (or run newgrp <groupname>) for the change to take effect in your current session.

4.10 Practical Permission Scenarios

Here are four real-world situations you'll encounter frequently, with the exact commands to handle them correctly.

Scenario 1: Making a script executable

```bash
$ touch backup.sh
$ echo '#!/bin/bash' > backup.sh
# Currently not executable:
$ ls -l backup.sh
-rw-r--r-- 1 ashik ashik 30 Jun 10 10:00 backup.sh
# Make it executable for the owner:
$ chmod u+x backup.sh
$ ls -l backup.sh
-rwxr--r-- 1 ashik ashik 30 Jun 10 10:00 backup.sh
# Now run it:
$ ./backup.sh
Backing up...
```

Scenario 2: Locking a sensitive file

```bash
# Config file with a password — should only be readable by owner:
$ chmod 600 db_config.env
$ ls -l db_config.env
-rw------- 1 ashik ashik 128 Jun 10 10:05 db_config.env
```

Scenario 3: Shared team directory with setgid and sticky bit

```bash
$ sudo groupadd devteam
$ sudo usermod -aG devteam ashik
$ sudo usermod -aG devteam alice
$ mkdir /shared/project
$ sudo chown root:devteam /shared/project
$ sudo chmod 2775 /shared/project   # setgid + rwxrwxr-x
$ sudo chmod +t  /shared/project    # sticky bit
$ ls -ld /shared/project
drwxrwsr-t 2 root devteam 4096 Jun 10 10:10 /shared/project
```

Scenario 4: Diagnosing a Permission denied error

```bash
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
# Check the file's permissions:
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1234 Jun 10 09:00 /etc/shadow
# You're not root and not in 'shadow' group — use sudo:
$ sudo cat /etc/shadow
```

4.11 Essential Commands: Quick Reference

Command	What It Does	Common Usage
whoami	Print your current username	whoami
id	Show UID, GID, and all group memberships	id ashik
groups	List groups a user belongs to	groups ashik
sudo	Run a command as root	sudo apt update
sudo -i	Open a root shell	sudo -i
sudo -l	List your sudo privileges	sudo -l

Command	What It Does	Common Usage
ls -l	Show files with permission strings	ls -la
chmod	Change file permissions	chmod 644 file.txt
chmod -R	Change permissions recursively	chmod -R 755 dir/
chown user file	Change file owner	sudo chown root file
chown user:grp file	Change owner and group	sudo chown ashik:dev app.py
chgrp grp file	Change file group	chgrp devteam app.py
adduser	Create a new user (interactive)	sudo adduser bob
usermod -aG grp user	Add user to a group	sudo usermod -aG sudo bob
passwd	Set or change a user's password	sudo passwd bob
deluser	Delete a user	sudo deluser bob
groupadd	Create a new group	sudo groupadd devteam
visudo	Safely edit the sudoers file	sudo visudo

4.12 Chapter Summary

✓ Linux is a **multi-user system**. Every file and process belongs to a user (UID) and a group (GID).

✓ The **root user** (UID 0) has unrestricted system access. Always prefer `sudo` over logging in as root.

✓ `sudo` runs one command as root, then returns to your normal user. It logs actions and requires your own password.

✓ Every file has **three permission types** (r/w/x) applied to **three categories**: owner (u), group (g), others (o).

✓ Permission strings like `-rwxr-xr-x` encode type + owner + group + others in 10 characters.

✓ `chmod` changes permissions in symbolic (u+x) or numeric octal (755) mode.

✓ `chown` changes file ownership. `chgrp` changes the group. Both require `sudo` for files you don't own.

✓ **Special bits**: setuid runs a file as its owner; setgid propagates group to new files; sticky bit prevents non-owners from deleting others' files.

✓ Memorise the common sets: `755` for executables/dirs, `644` for files, `600` for private files.

What's Coming in Chapter 5

With permissions mastered, Chapter 5 moves to one of Linux's greatest practical strengths: its software management system. You'll learn how to:

- Understand what a **package manager** is and why it's better than downloading installers
- Use `apt` to install, update, remove, and search for software
- Keep your entire system up to date with two commands
- Work with PPAs and third-party repositories
- Understand the difference between `apt` and `apt-get`
- Install software from source when no package is available

Chapter 5 is where Linux becomes genuinely faster than Windows or macOS for getting work done.

Author

Ashik A

github.com/ashihh07

This guide was created, organized, refined, and designed by Ashik A using GitHub documentation, industry best practices, publicly available learning resources, and AI-assisted research.